

React - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

REACT — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
TOPIC 1: JSX	5
1. Introduction	5
1.1 What is React, and why does it exist?	5
1.2 The exact problem React solves	5
2. What is JSX?	5
3. JSX Rules and Syntax	6
4. Real Life Analogy	6
5. Edge Cases	6
6. Frequently Asked Interview Questions	6
TOPIC 2: COMPONENTS	7
1. Introduction	7
2. Code Example — A Simple Functional Component	7
3. Component Composition — The Core React Mental Model	7
4. Real Life Analogy	7
5. Common Mistakes	8
6. Frequently Asked Interview Questions	8
TOPIC 3: PROPS	8
1. Introduction	8
2. Code Example	8
3. Props Are Immutable (Read-Only) — A Critical Rule	9
4. children — A Special Prop	9
5. Real Life Analogy	9
6. Edge Cases	9
7. Frequently Asked Interview Questions	9

TOPIC 4: STATE	10
1. Introduction	10
2. Code Example — useState	10
3. Why You MUST Use the Setter Function (Never Mutate State Directly)	10
4. State vs Props — Comparison Table	11
5. Real Life Analogy	11
6. Edge Cases	11
7. Common Mistakes	11
8. Frequently Asked Interview Questions	11
TOPIC 5: HOOKS	12
1. Introduction	12
2. useEffect — Running Side Effects (The Most Important Hook Beyond useState)	12
3. The Dependency Array — Critical, Frequently Misunderstood	12
4. Cleanup Functions — Preventing Memory Leaks	13
5. Custom Hooks — Reusing Stateful Logic	13
6. Rules of Hooks (Mandatory, Enforced by Linting Tools)	13
7. Real Life Analogy	13
8. Edge Cases	14
9. Common Mistakes	14
10. Frequently Asked Interview Questions	14
TOPIC 6: REACT ROUTER	14
1. Introduction	15
2. Code Example — Basic Routing Setup	15
3. <Link> vs a Plain <a> Tag — Why It Matters	15
4. Real Life Analogy	15
5. Frequently Asked Interview Questions	15
TOPIC 7: FORMS	16
1. Introduction	16
2. Code Example — A Controlled Form Calling the Spring Boot API	17
3. Controlled vs Uncontrolled Components	17

4. Real Life Analogy	17
5. Edge Cases	18
6. Frequently Asked Interview Questions	18

TOPIC 8: CONTEXT API **18**

1. Introduction	18
2. The Problem — "Prop Drilling"	19
3. The Fix — Context	19
4. Real Life Analogy	19
5. Common Mistakes	19
6. Frequently Asked Interview Questions	19

TOPIC 9: PERFORMANCE OPTIMIZATION **20**

1. Introduction	20
2. React.memo — Skip Re-rendering When Props Haven't Changed	20
3. useMemo — Cache an Expensive CALCULATION	20
4. useCallback — Cache a FUNCTION Reference	20
5. Real Life Analogy	20
6. Common Mistakes	21
7. Best Practices	21
8. Frequently Asked Interview Questions	21

TOPIC 10: ERROR BOUNDARIES **21**

1. Introduction	22
2. Code Example	22
3. What Error Boundaries DO and DON'T Catch	22
4. Real Life Analogy	22
5. Frequently Asked Interview Questions	23
6. Revision Notes (Entire React Section Consolidated)	23
7. Homework (Covers Entire React Section)	24
FINAL SUMMARY — REACT COMPLETE	24

REACT — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: JSX

1. Introduction

1.1 What is React, and why does it exist?

React is a JavaScript LIBRARY (not a full framework) for building USER INTERFACES — created by Facebook (2013) to solve a specific problem: as web applications grew more interactive (think Facebook's News Feed constantly updating), manually manipulating the DOM (Document Object Model — the browser's representation of a webpage) directly with plain JavaScript became SLOW, error-prone, and increasingly hard to reason about as an application grew.

1.2 The exact problem React solves

BEFORE REACT (manual DOM manipulation):

- Every time DATA changes (a new chat message arrives), you must MANUALLY find the right DOM element and update it: `document.getElementById(...).innerHTML = ...`
- As an app grows, tracking EXACTLY which DOM elements need updating for EVERY possible data change becomes a tangled, error-prone mess ("spaghetti code")
- Keeping the UI perfectly in SYNC with the underlying data becomes the developer's ENTIRE manual burden

WITH REACT:

- You describe WHAT the UI should look like for a GIVEN state of data (a "declarative" style), and React AUTOMATICALLY figures out the MINIMAL set of actual DOM updates needed when that data changes - you never manually touch the DOM yourself.

2. What is JSX?

JSX (JavaScript XML) is a syntax EXTENSION for JavaScript that lets you write HTML-LIKE markup DIRECTLY inside your JavaScript code — it's NOT a separate templating language (unlike JSP, from earlier in this course); it's SYNTACTIC SUGAR that gets COMPILED (via a tool called Babel) into regular JavaScript function calls.

```
// What YOU write (JSX):
const element = <h1 className="greeting">Hello, {userName}!</h1>;

// What it ACTUALLY compiles to (plain JavaScript):
const element = React.createElement("h1", { className: "greeting" }, "Hello, ", userName, "!");
```

3. JSX Rules and Syntax

```
function Greeting() {
  const name = "Alice";
  const isLoggedIn = true;

  return (
    <div className="container"> {/* className, NOT 'class' - 'class' is a reserved JS word! */}
    <h1>Hello, {name}!</h1> {/* {} embeds a JAVASCRIPT EXPRESSION directly in markup */}
    {isLoggedIn && <p>Welcome back!</p>} {/* conditional rendering using && */}
     {/* self-closing tags REQUIRED for elements with no children */}
    </div>
  );
}
```

Rule	Why
Must return ONE single root element (or a Fragment <code><>...</></code>)	JSX compiles to a single function call tree; multiple "sibling" top-level elements have no single root to compile into
<code>className</code> instead of <code>class</code>	<code>class</code> is a reserved keyword in JavaScript itself
<code>{expression}</code> embeds JavaScript	Lets you dynamically compute what to render, directly inline with the markup
Self-closing tags for childless elements (<code></code>)	JSX is stricter than HTML — unlike HTML, it REQUIRES proper closing

4. Real Life Analogy

- **Restaurant:** JSX is like a chef writing a plating **INSTRUCTION** directly alongside the dish's ingredients list, in **ONE** unified document, instead of keeping a **SEPARATE** plating diagram in an entirely different binder — combining "what data/ingredients go into this" with "exactly how it should visually look" in one readable place, rather than maintaining structure (HTML) and logic (JavaScript) as two disconnected, hard-to-keep-in-sync files.

5. Edge Cases

Case	Result
Returning MULTIPLE sibling elements with no wrapping tag	Compile error — wrap them in a single parent <code><div></code> or a <code><>...</></code> Fragment (which renders NO extra DOM element at all)
Using <code>class</code> instead of <code>className</code>	Doesn't cause an error, but the CSS class won't actually apply — React specifically looks for <code>className</code>
Rendering a plain JavaScript object directly inside <code>{}</code>	Runtime error — React can render strings/numbers/elements/arrays, but not raw objects directly

6. Frequently Asked Interview Questions

- 1 **What is React?** A JavaScript library for building user interfaces, using a declarative, component-based approach instead of manual DOM manipulation.
- 2 **What is JSX?** A syntax extension letting you write HTML-like markup directly in JavaScript, compiled (via Babel) into `React.createElement()` calls — not a separate templating language.

- 3 **Why does JSX use `className` instead of `class`?** Because `class` is a reserved keyword in JavaScript itself.
 - 4 **Why must a JSX expression return only ONE root element?** JSX compiles into a single nested function-call tree; multiple top-level sibling elements have no single root to represent that tree.
-
-

TOPIC 2: COMPONENTS

1. Introduction

What is it? A component is a REUSABLE, SELF-CONTAINED piece of UI — a JavaScript FUNCTION that returns JSX describing what should appear on screen. React applications are built by COMPOSING many small components together into larger ones, exactly analogous to how you compose classes/methods in Java, or how MVC (Advanced Java) separates concerns into distinct pieces.

2. Code Example — A Simple Functional Component

```
function EmployeeCard({ name, department, salary }) { // a "functional component" - just a JS function!
  return (
    <div className="employee-card">
      <h3>{name}</h3>
      <p>Department: {department}</p>
      <p>Salary: ${salary.toLocaleString()}</p>
    </div>
  );
}

function EmployeeList() { // a component COMPOSING other components
  return (
    <div>
      <EmployeeCard name="Alice" department="Engineering" salary={75000} />
      <EmployeeCard name="Bob" department="Sales" salary={60000} />
    </div>
  );
}
```

3. Component Composition — The Core React Mental Model

```
App
  ■■■ EmployeeList
  ■■■ EmployeeCard (Alice)
  ■■■ EmployeeCard (Bob)
  ■■■ EmployeeCard (Carol)
```

Each component is SELF-CONTAINED and REUSABLE - the SAME `EmployeeCard` component definition renders DIFFERENT output depending on what data (props, Topic 3) it's given, exactly like a Java class template producing different OBJECT instances with different field values.

4. Real Life Analogy

- **Restaurant:** A component is like a STANDARDIZED recipe card for ONE specific dish ("Caesar Salad") — the SAME recipe card is reused for EVERY customer's order, just with the SPECIFIC ingredients/portion sizes (the data/props) varying per order. A full menu (the App) is composed of MANY such individual, reusable recipe cards combined together.

5. Common Mistakes

- Building ONE giant, monolithic component handling EVERYTHING, instead of breaking the UI into small, focused, reusable pieces — mirrors the exact same "keep classes/methods focused" principle from Core Java's OOP topics.
- Not recognizing that a component is JUST a function — overcomplicating the mental model unnecessarily.

6. Frequently Asked Interview Questions

- 1 **What is a React component?** A reusable, self-contained JavaScript function that returns JSX describing a piece of UI.
- 2 **How are complex React UIs typically built?** By COMPOSING many small, focused components together into larger ones — the same "small, focused units" principle found in well-designed Java classes/methods.
- 3 **Can the same component be reused with different data?** Yes — that's the entire point; a component is a reusable TEMPLATE, and the specific data it renders comes from props (next topic).

TOPIC 3: PROPS

1. Introduction

What is it? Props ("properties") are how DATA is passed FROM a parent component INTO a child component — READ-ONLY from the child's perspective (a child can NEVER modify its own props directly), directly analogous to passing arguments into a Java method or constructor.

2. Code Example

```
function EmployeeCard({ name, department, salary }) { // DESTRUCTURING props directly in the function signature
  return (
    <div>
      <h3>{name}</h3>
      <p>{department}</p>
    </div>
  );
}

// The PARENT passes props like HTML attributes:
<EmployeeCard name="Alice" department="Engineering" salary={75000} />
```


3. Props Are Immutable (Read-Only) — A Critical Rule

```
function EmployeeCard({ salary }) {  
  salary = salary * 1.1; // NEVER do this - mutating props directly VIOLATES React's core data-flow model  
  return <p>{salary}</p>;  
}
```

React's data flow is deliberately **UNIDIRECTIONAL** ("one-way data binding") — data flows DOWN from parent to child via props, and NEVER flows back up by a child directly mutating what it received. If a child needs to communicate something back UP to its parent (e.g., "the user clicked this button"), it does so by calling a FUNCTION passed down as a prop (see the Forms topic for a concrete example).

4. `children` — A Special Prop

```
function Card({ children }) {  
  return <div className="card">{children}</div>;  
}  
  
// Usage - anything BETWEEN the opening/closing tags becomes the 'children' prop automatically:  
<Card>  
  <h2>Title</h2>  
  <p>Some content here.</p>  
</Card>
```

5. Real Life Analogy

- **Restaurant:** Props are like the SPECIFIC order details a waiter hands to the kitchen for ONE particular dish ("no onions, extra spicy") — the kitchen (child component) READS and ACTS on these instructions but never reaches back and changes the CUSTOMER'S original order slip (the parent's data) directly; any request to CHANGE something goes back through the proper channel (calling a function passed down, not direct mutation).

6. Edge Cases

Case	Result
A required prop not provided by the parent	The child receives <code>undefined</code> for that prop — no automatic error (unless using TypeScript, covered in a later section, or explicit prop-type validation)
Attempting to reassign a destructured prop variable	Doesn't throw an error in plain JavaScript, but VIOLATES React's data flow model and can cause subtle, hard-to-debug UI inconsistencies

7. Frequently Asked Interview Questions

- 1 **What are props?** Data passed from a parent component into a child component, read-only from the child's perspective.
- 2 **Why is React's data flow described as "unidirectional"?** Data only flows DOWNWARD (parent to child) via props; a child never directly mutates the data it received — any "upward" communication happens via a callback function passed down as a prop.

- 3 **What is the special `children` prop?** It automatically captures whatever JSX is nested BETWEEN a component's opening and closing tags in its usage.
 - 4 **Can a child component modify its own props directly?** No — props are read-only; attempting to mutate them directly violates React's core data model.
-

TOPIC 4: STATE

1. Introduction

What is it? State is data that a component **MANAGES ITSELF** and that **CAN CHANGE OVER TIME** (e.g., a form input's current value, whether a modal is open) — unlike props (passed IN from outside), state is **INTERNAL** to a component, and **CHANGING** it automatically triggers React to **RE-RENDER** that component (and its children) to reflect the new value.

2. Code Example — `useState`

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // useState(initialValue) returns [currentValue,
  updaterFunction]

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

3. Why You MUST Use the Setter Function (Never Mutate State Directly)

```
// WRONG - directly mutating the state variable does NOT trigger a re-render!
count = count + 1;

// CORRECT - calling the setter function tells React "this changed, please re-render"
setCount(count + 1);
```

React only knows to **RE-RENDER** a component when you call its **STATE SETTER FUNCTION** — directly reassigning the variable bypasses React's tracking entirely, and the UI simply won't update, even though the underlying JavaScript value technically changed.

4. State vs Props — Comparison Table

Aspect	Props	State
Where it comes from	Passed IN from a PARENT component	Managed INTERNALLY by the component itself
Mutability	Read-only (immutable from the child's view)	Can be updated via its setter function
Who can change it	Only the PARENT (by re-rendering with new prop values)	The component itself (or, if passed down, a designated handler function)
Triggers re-render when changed?	Yes (when the PARENT re-renders with new values)	Yes (when its OWN setter function is called)

5. Real Life Analogy

- **Restaurant:** State is like a SPECIFIC customer's OWN evolving order status ("still deciding," "order placed," "food served") that THEY (or their assigned waiter) actively update as things progress — unlike props, which are more like FIXED instructions handed to them ONCE by an outside source (the parent) at the start.

6. Edge Cases

Case	Result
Calling the setter function with the SAME value it already has	React is smart enough to SKIP a genuinely unnecessary re-render in many cases (an optimization, not a strict guarantee)
Updating state based on the PREVIOUS state value in rapid succession	Use the FUNCTIONAL update form: <code>setCount(prev => prev + 1)</code> — guarantees you're building on the LATEST value, avoiding subtle bugs from React's asynchronous, batched state updates

7. Common Mistakes

- Directly mutating a state variable instead of calling its setter function — the UI silently fails to update.
- Not using the functional update form (`setCount(prev => prev + 1)`) when the new state depends on the PREVIOUS state, especially in rapid/batched updates.

8. Frequently Asked Interview Questions

- 1 **What is state in React?** Data a component manages internally that can change over time, with changes triggering React to re-render the component to reflect the new value.
- 2 **What does `useState` return?** An array with exactly two elements: the current state value, and a function to update it.
- 3 **Why must you use the setter function instead of directly reassigning the state variable?** React only re-renders in response to the setter function being called — directly mutating the variable bypasses React's change-tracking entirely, and the UI won't update.

- 4 **When should you use the functional update form** (`setCount(prev => prev + 1)`)? When the new state value depends on the PREVIOUS state, especially with rapid/batched updates, to guarantee you're building on the most current value rather than a potentially stale one.

TOPIC 5: HOOKS

1. Introduction

What is it? Hooks are special functions (always starting with `use`) that let a FUNCTIONAL component "hook into" React features — state (`useState`, Topic 4), side effects (`useEffect`), and more — introduced in React 16.8 (2019) to REPLACE the older, more verbose "class component" style entirely.

2. `useEffect` — Running Side Effects (The Most Important Hook Beyond `useState`)

What is it? `useEffect` lets you run code in response to a component RENDERING or specific values CHANGING — the standard place for API calls, subscriptions, timers, or any interaction with something OUTSIDE React's own rendering system.

```
import { useState, useEffect } from "react";

function EmployeeProfile({ employeeId }) {
  const [employee, setEmployee] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);
    fetch(`http://localhost:8080/api/employees/${employeeId}`) // calling the Spring Boot REST API!
      .then(response => response.json())
      .then(data => {
        setEmployee(data);
        setLoading(false);
      });
  }, [employeeId]); // DEPENDENCY ARRAY - re-run this effect ONLY when 'employeeId' changes

  if (loading) return <p>Loading...</p>;
  return <h2>{employee.firstName} {employee.lastName}</h2>;
}
```

3. The Dependency Array — Critical, Frequently Misunderstood

```
useEffect(() => { /* runs on EVERY render */ }); // NO dependency array
useEffect(() => { /* runs ONCE, after the FIRST render only */ }, []); // EMPTY dependency array
useEffect(() => { /* runs after first render AND whenever employeeId changes */ }, [employeeId]);
```

Dependency Array	Behavior
Omitted entirely	Effect runs after EVERY single render — rarely what you actually want

Dependency Array	Behavior
<code>[]</code> (empty)	Effect runs ONCE, only after the initial render — perfect for "fetch data on component mount"
<code>[value1, value2]</code>	Effect re-runs whenever ANY listed value CHANGES between renders

4. Cleanup Functions — Preventing Memory Leaks

```
useEffect(() => {
  const timer = setInterval(() => console.log("tick"), 1000);

  return () => clearInterval(timer); // CLEANUP function - runs when the component UNMOUNTS
  // (or before the effect re-runs) - prevents the timer from
  // leaking/continuing after this component is gone
}, []);
```

5. Custom Hooks — Reusing Stateful Logic

```
function useEmployee(employeeId) { // a CUSTOM hook - just a regular function using OTHER hooks!
  const [employee, setEmployee] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(`http://localhost:8080/api/employees/${employeeId}`)
      .then(res => res.json())
      .then(data => { setEmployee(data); setLoading(false); });
  }, [employeeId]);

  return { employee, loading };
}

// NOW reusable across MULTIPLE components, exactly like extracting a shared method in Java:
function EmployeeProfile({ employeeId }) {
  const { employee, loading } = useEmployee(employeeId);
  if (loading) return <p>Loading...</p>;
  return <h2>{employee.firstName}</h2>;
}
```

6. Rules of Hooks (Mandatory, Enforced by Linting Tools)

- 1 **Only call hooks at the TOP LEVEL** of a component/custom hook — never inside loops, conditions, or nested functions.
- 2 **Only call hooks from React function components or other custom hooks** — never from a plain regular JavaScript function.

Why this rule exists: React tracks hooks by the ORDER they're called in, across renders — calling them conditionally could change that order between renders, corrupting React's internal state tracking entirely.

7. Real Life Analogy

- **Restaurant:** `useEffect` is like a chef's STANDING INSTRUCTION triggered automatically whenever a SPECIFIC condition changes ("whenever the customer's TABLE NUMBER changes, re-fetch THEIR specific order details") — rather than the chef needing to manually remember to re-check every single time something might have changed.

8. Edge Cases

Case	Result
Forgetting the dependency array on a <code>useEffect</code> that fetches data	The fetch runs on EVERY render — potentially an infinite loop if the effect ITSELF also triggers a state update causing a re-render
Calling a hook conditionally (<code>if (x) { useState(...) }</code>)	Violates the Rules of Hooks — causes genuinely broken, unpredictable behavior, flagged as an ERROR by React's linting rules

9. Common Mistakes

- Forgetting the dependency array entirely, causing unintended infinite re-fetch loops.
- Forgetting a cleanup function for subscriptions/timers, causing memory leaks as components unmount.
- Calling hooks conditionally or inside loops, violating the Rules of Hooks.

10. Frequently Asked Interview Questions

- 1 **What is `useEffect` used for?** Running side effects (API calls, subscriptions, timers) in response to a component rendering or specific values changing.
- 2 **What does an EMPTY dependency array (`[]`) mean for `useEffect`?** The effect runs exactly ONCE, right after the component's initial render, and never again (unless the component unmounts and remounts).
- 3 **What is a cleanup function in `useEffect`, and when does it run?** A function RETURNED from the effect, run when the component unmounts (or before the effect re-runs), used to prevent memory leaks from subscriptions/timers.
- 4 **What are the two Rules of Hooks?** Only call hooks at the top level (never in loops/conditions), and only call them from React components or other custom hooks.
- 5 **What is a custom hook?** A reusable function (starting with `use`) that itself uses other hooks, letting you extract and share stateful logic across multiple components.

TOPIC 6: REACT ROUTER

1. Introduction

What is it? React Router is the standard library for handling NAVIGATION in a React application — since React apps are typically **Single-Page Applications (SPAs)** (the browser loads ONE HTML page, and React dynamically swaps content WITHOUT full page reloads), React Router manages URL-to-component mapping entirely on the CLIENT side.

2. Code Example — Basic Routing Setup

```
import { BrowserRouter, Routes, Route, Link, useNavigate, useParams } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/employees">Employees</Link>
      </nav>

      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/employees" element={<EmployeeListPage />} />
        <Route path="/employees/:id" element={<EmployeeDetailPage />} /> {/* dynamic route parameter */}
        <Route path="*" element={<NotFoundPage />} /> {/* catch-all for unmatched routes */}
      </Routes>
    </BrowserRouter>
  );
}

function EmployeeDetailPage() {
  const { id } = useParams(); // reads the ":id" from the CURRENT URL
  const navigate = useNavigate(); // PROGRAMMATIC navigation (e.g., after a form submits)

  return (
    <div>
      <h2>Employee #{id}</h2>
      <button onClick={() => navigate("/employees")}>Back to list</button>
    </div>
  );
}
```

3. `<Link>` vs a Plain `<a>` Tag — Why It Matters

```
<a href="/employees">Employees</a> {/* triggers a FULL PAGE RELOAD - loses all React state! */}
<Link to="/employees">Employees</Link> {/* CLIENT-SIDE navigation - NO page reload, React state preserved */}
```

4. Real Life Analogy

- **Restaurant:** React Router is like a restaurant's internal SIGNAGE system directing customers to different SECTIONS (dining room, bar, restrooms) WITHOUT them ever needing to walk back OUTSIDE and re-enter through the front door each time (a full page reload) — you move smoothly between areas within the SAME continuous visit.

5. Frequently Asked Interview Questions

- 1 **What is React Router?** The standard library for client-side navigation/routing in a React Single-Page Application, mapping URLs to components without full page reloads.
 - 2 **Why use `<Link>` instead of a plain `<a>` tag for internal navigation?** `<a>` triggers a full page reload (losing all current React state); `<Link>` performs client-side navigation, keeping the SPA's state intact.
 - 3 **What does `useParams()` do?** Reads dynamic segments (like `:id`) from the current URL.
-

TOPIC 7: FORMS

1. Introduction

What is it? Forms in React are typically built as **CONTROLLED COMPONENTS** — where React's STATE is the SINGLE SOURCE OF TRUTH for an input's current value, rather than letting the browser's own DOM manage it independently, giving you full programmatic control over validation, formatting, and submission.

2. Code Example — A Controlled Form Calling the Spring Boot API

```
import { useState } from "react";

function CreateEmployeeForm() {
  const [formData, setFormData] = useState({ firstName: "", salary: "" });
  const [error, setError] = useState(null);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value })); // spread + update ONE field, keeping others
    unchanged
  };

  const handleSubmit = async (e) => {
    e.preventDefault(); // PREVENTS the browser's default full-page-reload submit
    try {
      const response = await fetch("http://localhost:8080/api/employees", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(formData),
      });
      if (!response.ok) {
        const err = await response.json();
        setError(err.message);
        return;
      }
      alert("Employee created successfully!");
    } catch (err) {
      setError("Network error - please try again.");
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="firstName" value={formData.firstName} onChange={handleChange} placeholder="First Name"
      />
      <input name="salary" value={formData.salary} onChange={handleChange} placeholder="Salary"
      type="number" />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Create Employee</button>
    </form>
  );
}
```

3. Controlled vs Uncontrolled Components

Aspect	Controlled	Uncontrolled
Source of truth	React STATE drives the input's value	The DOM itself holds the value (accessed via a "ref" only when needed)
Real-time validation/formatting	Easy — you see every keystroke via state updates	Harder — you only read the value at specific moments (e.g., on submit)
Recommendation	The STANDARD, default approach for most forms	Occasionally used for simple, uncomplicated forms or integrating non-React libraries

4. Real Life Analogy

- **Bank:** A controlled form is like a bank teller who WATCHES and RECORDS every single digit you type into a form IN REAL TIME, immediately catching and flagging an invalid entry as you go — versus an uncontrolled form, which is like simply COLLECTING the completed form at the very end and only checking it then.

5. Edge Cases

Case	Result
Forgetting <code>e.preventDefault()</code> in the submit handler	The browser performs its OWN default form submission behavior (a full page reload/navigation), bypassing your React logic entirely
An <code><input></code> with a <code>value</code> prop but NO <code>onChange</code> handler	React treats it as READ-ONLY and logs a console warning — a controlled input MUST have both

6. Frequently Asked Interview Questions

- 1 **What is a "controlled component" in React forms?** An input whose current value is driven entirely by React state, rather than the DOM managing it independently — updated via an `onChange` handler on every keystroke.
- 2 **Why must you call `e.preventDefault()` in a form's submit handler?** To stop the browser's DEFAULT full-page-reload form submission behavior, allowing your React `handleSubmit` logic (like an `async fetch` call) to run instead.
- 3 **What warning does React give for an input with a `value` prop but no `onChange` handler?** It's treated as read-only, and React logs a console warning, since a controlled input needs both to function as an editable field.

TOPIC 8: CONTEXT API

1. Introduction

What is it? The Context API lets you share DATA across MANY components WITHOUT manually passing it down through every single intermediate component's props — solving the "prop drilling" problem, where deeply nested components need data from a distant ancestor.

2. The Problem — "Prop Drilling"

```
// WITHOUT Context: 'user' must be passed through EVERY intermediate component,  
// even ones that don't actually need it themselves, just to reach the deeply nested one:  
function App() {  
  const user = { name: "Alice" };  
  return <Layout user={user} />;  
}  
function Layout({ user }) { return <Sidebar user={user} />; } // doesn't use 'user' itself, just  
PASSES it along  
function Sidebar({ user }) { return <UserProfile user={user} />; } // same - just passing through  
function UserProfile({ user }) { return <p>{user.name}</p>; } // FINALLY actually uses it
```

3. The Fix — Context

```
import { createContext, useContext, useState } from "react";  
  
const UserContext = createContext(null); // create the context  
  
function App() {  
  const [user] = useState({ name: "Alice" });  
  return (  
    <UserContext.Provider value={user}> {/* makes 'user' available to ALL descendants */}  
    <Layout />  
    </UserContext.Provider>  
  );  
}  
function Layout() { return <Sidebar />; } // NO NEED to pass 'user' through anymore!  
function Sidebar() { return <UserProfile />; }  
function UserProfile() {  
  const user = useContext(UserContext); // read directly from context, skipping the "drilling"  
  return <p>{user.name}</p>;  
}
```

4. Real Life Analogy

- **Restaurant:** Prop drilling is like a message needing to be PERSONALLY relayed by every single staff member standing between the manager and the specific chef who actually needs it, even though most of those staff have nothing to do with that message. Context is like a restaurant-wide ANNOUNCEMENT SYSTEM — the manager posts it ONCE, and ANY staff member who needs that information can check it directly, without a long human chain relaying it step by step.

5. Common Mistakes

- Using Context for EVERY piece of shared state, even simple parent-child relationships where regular props would be simpler and clearer — Context is best reserved for genuinely widely-needed, deeply-shared data (current user, theme, locale).
- Not realizing that EVERY consumer of a Context re-renders whenever the Context's value changes — for very frequently-changing data, this can become a performance concern (Topic 9).

6. Frequently Asked Interview Questions

- 1 **What problem does the Context API solve?** "Prop drilling" — passing data through many intermediate components that don't actually need it themselves, just to reach a deeply nested descendant that does.
 - 2 **What are the two main pieces of the Context API?** `createContext()` (creates the context) and `useContext()` (reads its current value from within a component).
 - 3 **When should you AVOID using Context?** For simple, shallow parent-child data passing where regular props are clearer, or for VERY frequently-changing values, since every consumer re-renders on each context value change.
-

TOPIC 9: PERFORMANCE OPTIMIZATION

1. Introduction

What is it? React re-renders a component whenever its state OR its parent's state changes — for most apps this is fast enough to be invisible, but for LARGE lists or EXPENSIVE computations, UNNECESSARY re-renders can cause visible UI lag. React provides specific tools to avoid this WASTED work.

2. `React.memo` — Skip Re-rendering When Props Haven't Changed

```
const EmployeeCard = React.memo(function EmployeeCard({ name, salary }) {  
  console.log("Rendering:", name); // with React.memo, this only logs when name/salary ACTUALLY change  
  return <div>{name}: ${salary}</div>;  
});
```

3. `useMemo` — Cache an Expensive CALCULATION

```
function EmployeeStats({ employees }) {  
  const totalSalary = useMemo(() => {  
    console.log("Recalculating total..."); // only logs when 'employees' ACTUALLY changes  
    return employees.reduce((sum, e) => sum + e.salary, 0);  
  }, [employees]); // dependency array - SAME concept as useEffect!  
  
  return <p>Total: ${totalSalary}</p>;  
}
```

4. `useCallback` — Cache a FUNCTION Reference

```
function EmployeeList({ employees }) {  
  const handleDelete = useCallback((id) => {  
    console.log("Deleting employee:", id);  
  }, []); // the SAME function reference is reused across re-renders,  
  // preventing child components (wrapped in React.memo) from  
  // unnecessarily re-rendering just because they received a  
  // "new" (but functionally identical) function prop each time  
  return employees.map(e => <EmployeeCard key={e.id} onDelete={handleDelete} {...e} />);  
}
```

5. Real Life Analogy

- **Restaurant:** `useMemo` is like a chef CACHING the result of a complex sauce reduction that takes an hour to prepare — only actually redoing that expensive process when the INGREDIENTS genuinely change, rather than needlessly redoing it from scratch for every single dish that happens to use it. `React.memo` is like a kitchen STATION that only actually re-cooks a dish when the SPECIFIC order details for THAT dish change, ignoring unrelated activity happening elsewhere in the kitchen.

6. Common Mistakes

- Wrapping EVERY component in `React.memo/useMemo/useCallback` by default — these optimizations have their OWN overhead (comparing values/dependencies); apply them DELIBERATELY where profiling shows an ACTUAL performance problem, not as a blanket habit.
- Forgetting the dependency array on `useMemo/useCallback`, causing the cached value/function to NEVER update when it actually should.

7. Best Practices

- Profile FIRST (using React DevTools' Profiler) to identify GENUINE performance bottlenecks before reaching for these optimization tools — premature optimization adds complexity without proven benefit.
- Use `key` props correctly on list items (a STABLE, UNIQUE identifier like a database ID, never the array INDEX for lists that can reorder/change) to help React efficiently track which items changed.

8. Frequently Asked Interview Questions

- 1 **What does `React.memo` do?** Wraps a component so it SKIPS re-rendering if its props haven't actually changed since the last render.
- 2 **What's the difference between `useMemo` and `useCallback`?** `useMemo` caches the RESULT of an expensive calculation; `useCallback` caches a FUNCTION reference itself, both re-computed only when their dependency array changes.
- 3 **Should you wrap every component/function in these optimization hooks by default?** No — they have their own overhead; apply them deliberately where profiling shows an actual performance issue, not as a blanket default habit.
- 4 **Why shouldn't you use an array's index as the `key` prop for a reorderable/changeable list?** React uses `key` to track which specific items changed/moved/were removed between renders; using the index breaks this tracking when the list's order or contents change, potentially causing incorrect UI updates or lost component state.

TOPIC 10: ERROR BOUNDARIES

1. Introduction

What is it? An Error Boundary is a SPECIAL React component that CATCHES JavaScript errors occurring ANYWHERE in its CHILD component tree, logs them, and displays a FALLBACK UI — instead of the ENTIRE application crashing to a blank white screen, exactly analogous to a try-catch block (Core Java) or a Spring Boot `@RestControllerAdvice` global exception handler, but for the UI RENDERING process itself.

2. Code Example

```
import { Component } from "react";

class ErrorBoundary extends Component { // Error Boundaries currently MUST be class components
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true }; // triggers the fallback UI on the NEXT render
  }

  componentDidCatch(error, errorInfo) {
    console.error("Caught an error:", error, errorInfo); // log it (e.g., to a monitoring service)
  }

  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong. Please try refreshing the page.</h2>;
    }
    return this.props.children; // render children NORMALLY if no error occurred
  }
}

// Usage - wrap ANY part of your component tree you want protected:
function App() {
  return (
    <ErrorBoundary>
      <EmployeeList />
    </ErrorBoundary>
  );
}
```

3. What Error Boundaries DO and DON'T Catch

Catches	Does NOT Catch
Errors thrown during RENDERING	Errors inside event handlers (<code>onClick</code> , etc. — use a regular try-catch there instead)
Errors in lifecycle methods	Errors in ASYNCHRONOUS code (<code>setTimeout</code> , a fetch callback)
Errors in constructors of the tree below it	Errors in the Error Boundary component ITSELF

4. Real Life Analogy

- **Restaurant:** An Error Boundary is like a restaurant having a CONTAINED, SEPARATE fire-suppression system for EACH individual cooking station — if ONE station has a fire (a component crashes), it's

CONTAINED and handled locally, WITHOUT shutting down and evacuating the ENTIRE restaurant (crashing the whole application) — customers at OTHER, unaffected tables continue being served normally.

5. Frequently Asked Interview Questions

- 1 **What is an Error Boundary?** A special component that catches JavaScript errors occurring in its child component tree during rendering, displaying a fallback UI instead of crashing the entire application.
- 2 **What kinds of errors does an Error Boundary NOT catch?** Errors inside event handlers, asynchronous code (like `setTimeout/fetch` callbacks), and errors within the Error Boundary component itself.
- 3 **What's the conceptual parallel to Error Boundaries from earlier in this course?** A `try-catch` block (Core Java) or Spring Boot's `@RestControllerAdvice` (centralized exception handling) — all provide a "contain the failure, don't let it crash everything" mechanism, just at different layers (a single operation, an entire backend API, or here, the UI rendering tree).

6. Revision Notes (Entire React Section Consolidated)

```
REACT — CHEAT SHEET
=====
REACT solves: manual DOM manipulation becoming unmanageable at scale -
DECLARATIVE ("describe the UI for given data") instead of imperative

JSX: HTML-like syntax compiled to React.createElement() calls
className (not class), {} embeds JS expressions, ONE root element required

COMPONENTS: reusable functions returning JSX, composed together (like Java
classes/methods)
PROPS: read-only data passed PARENT -> CHILD (unidirectional data flow!)
NEVER mutate props directly
STATE (useState): component-OWNED data that changes over time; ALWAYS use
the setter function, NEVER mutate directly; use functional updates
(prev => ...) when new state depends on old state

HOOKS: useEffect (side effects: API calls, subscriptions) + dependency array
([] = once on mount, [x] = re-run when x changes, omitted = every render)
+ cleanup function (return () => ...) prevents leaks
Rules: top-level only, React functions/hooks only
Custom hooks: reusable stateful logic, just regular functions using other hooks

REACT ROUTER: <Link> (client-side, no reload) vs <a> (full reload, loses
state), useParams() (URL segments), useNavigate() (programmatic nav)

FORMS: CONTROLLED components (state = single source of truth), onChange +
value, e.preventDefault() in submit handler

CONTEXT API: solves "prop drilling" - createContext() + Provider + useContext()
- don't overuse for simple/shallow or very-frequently-changing data

PERFORMANCE: React.memo (skip re-render if props unchanged), useMemo (cache
a VALUE), useCallback (cache a FUNCTION reference) - apply DELIBERATELY,
after profiling, not by default everywhere
key prop: stable unique ID, NEVER array index for reorderable lists

ERROR BOUNDARIES: class component catching render-time errors in children,
shows fallback UI - does NOT catch event handler or async errors
```

7. Homework (Covers Entire React Section)

- 1 Build an `EmployeeList` app: fetch data from your Spring Boot `/api/employees` endpoint using `useEffect` + `fetch`, rendering each employee via a reusable `EmployeeCard` component.
 - 2 Add React Router with a list page and a detail page (`/employees/:id`), navigating between them via `<Link>` and reading the ID via `useParams()`.
 - 3 Build a controlled `CreateEmployeeForm` that POSTs to your API, displaying validation errors returned by your Spring Boot `@RestControllerAdvice` handler.
 - 4 Wrap your app in an `ErrorBoundary`, then deliberately throw an error inside a child component's render to verify the fallback UI appears instead of a blank crashed page.
-

FINAL SUMMARY — REACT COMPLETE

This concludes the **React** section — 10 topics:

- 1 **JSX** — HTML-like syntax compiled to JavaScript function calls.
- 2 **Components** — reusable, composable UI functions.
- 3 **Props** — read-only, unidirectional parent-to-child data flow.
- 4 **State** — internal, mutable-via-setter component data (`useState`).
- 5 **Hooks** — `useEffect` for side effects, custom hooks for reusable logic, the Rules of Hooks.
- 6 **React Router** — client-side SPA navigation.
- 7 **Forms** — controlled components, calling your Spring Boot API directly.
- 8 **Context API** — solving prop drilling.
- 9 **Performance Optimization** — `React.memo/useMemo/useCallback`, applied deliberately.
- 10 **Error Boundaries** — containing UI crashes gracefully.

You now have a complete, practical foundation for building the frontend that consumes every REST API you built across the Spring Boot sections of this course. Next: JavaScript, formalizing the language fundamentals underlying everything in React.

(END OF REACT MASTER NOTES — ready to proceed to JavaScript whenever you say "Next Topic.")